

SPRINT

Sprint 1 — Core API Reliability

Weeks 1–2

Sprint Goal: *Make Priority Management a trustworthy backend capability.*

This starts from where you are **today**, not from scratch.

Committed stories:

PRI-04 — Partial update 🟡

Finish PATCH behavior and make sure the stored object remains a complete Priority.

PRI-05 — Delete priority

Finalize the response contract and test success + missing ID.

PRI-06 — Priority validation

This is a major learning story. Establish constraints for category, importance, status, URLs, IDs, required strings, etc.

ENG-01A — Priority API tests

Not the entire application's test suite. Just Priority.

Acceptance at the sprint level:

GET all	✓
GET one	✓
POST	✓
PATCH	✓
DELETE	✓
Valid input	✓
Invalid input	✓
Missing resources	✓
Response models	✓
Status codes	✓
Automated tests	✓

Demo: Run automated tests, open /docs, create a Priority, update it, retrieve it and delete it.

That's a real sprint demo.

Sprint 2 — Persistent Data

Weeks 3–4

Sprint Goal: *JalenOS remembers.*

This is probably your largest **learning sprint**.

Stories:

DATA-01 — Persist priorities

Introduce the database.

DATA-02 — Separate data-access/business logic

Your route shouldn't eventually contain everything.

DATA-03 — Schema migrations

ENG-02 — Environment configuration

Then migrate your existing Priority CRUD away from:

```
real_priorities = [...]
```

to persistent storage.

Demo:

Create priorities → shut FastAPI completely down → restart → retrieve priorities → **they're still there.**

That's a huge milestone.

This sprint will introduce concepts worth researching properly: relational databases, ORM concepts, SQLAlchemy, sessions, transactions, migrations, environment variables.

We'll give you beginner + production resources as each story enters **In Progress**, rather than dumping 40 tutorials on you now.

Sprint 3 — Priority Intelligence

Weeks 5–6

Sprint Goal: *JalenOS stops merely storing priorities and starts making decisions.*

Stories:

INT-01 — Priority scoring

Design the first algorithm.

Potential inputs:

Importance
Urgency
Deadline proximity
Status
Domain

Then:

INT-02 — Top-priority selection

JalenOS can answer:

What should Jalen work on right now?

I'd also introduce:

PROJ-01 — Track projects

because Projects can become another source of priorities and give us richer test data.

And potentially begin:

INV-01 — Investment watchlist

if Sprint 3 capacity permits.

Demo:

Create several competing priorities across Career, Projects, Investments, etc. → JalenOS scores them → returns them ranked → identifies #1 → changing an input causes ranking to change predictably.

This is also where your interest in algorithms begins becoming directly relevant.

Sprint 4 — JalenOS Experience

Weeks 7–8

Sprint Goal: *Turn the backend capabilities into something you actually want to open.*

Now Angular becomes a major focus.

Stories:

DASH-03 — Application shell/navigation

START-01 — Start Screen

START-02 — Start → Dashboard navigation

DASH-01 — Domain overview

DASH-02 — Priority overview

EXT-01 — Weather

NOTE-01 — Quick note capture, if capacity permits.

This is when your earlier wireframe work comes back.

The Start Screen finally becomes something like:

```
7:42 PM

"Quote of the day..."

TOP PRIORITY
Continue JalenOS Alpha

Kansas City • 79°F

CURRENT BOOK
Grokking Algorithms

↓
```

Then enter the Dashboard.

Demo: Open JalenOS in a browser instead of Swagger and actually use it.

This will probably be the sprint where you first think:

Holy shit, this is actually JalenOS.

Sprint 5 — Career Workflow

Weeks 9–10

Sprint Goal: *Prove that a JalenOS domain can feed intelligence back into the Core.*

Stories:

CAR-01 — Companies

CAR-02 — Opportunities

CAR-03 — Applications

CAR-05 — Follow-ups

And the crucial integration:

```
Career
  ↓
Application hasn't received follow-up
  ↓
Follow-up becomes due
  ↓
JalenOS recognizes actionable event
  ↓
Priority system
  ↓
Potentially appears as Top Priority
```

That's much more interesting than merely building a job tracker.

It demonstrates your core architectural thesis:

Domains feed information into a central decision-support system.

Demo: Add company → add job → record application → create/update follow-up → JalenOS surfaces the resulting action.

Sprint 6 — Alpha Release

Weeks 11–12

This is where we need discipline.

Sprint Goal: *Ship JalenOS Alpha.*

Not:

"Build every remaining idea."

Investment work gets a constrained vertical slice.

Target:

INV-01 — Watchlist (*if not Sprint 3*)

INV-02 — Market data

Then one quantitative capability.

I'd choose:

INV-04 — Efficient Frontier integration

But we will have to decompose that XL story before Sprint 6. If it looks too large by Week 10, we substitute a smaller quantitative analysis rather than jeopardizing the release.

Then production:

ENG-03 — Logging if capacity allows

ENG-05 — Security fundamentals

ENG-06 — CI pipeline

ENG-07 — Deployment

And:

Bug fixing / integration / UI cleanup / documentation